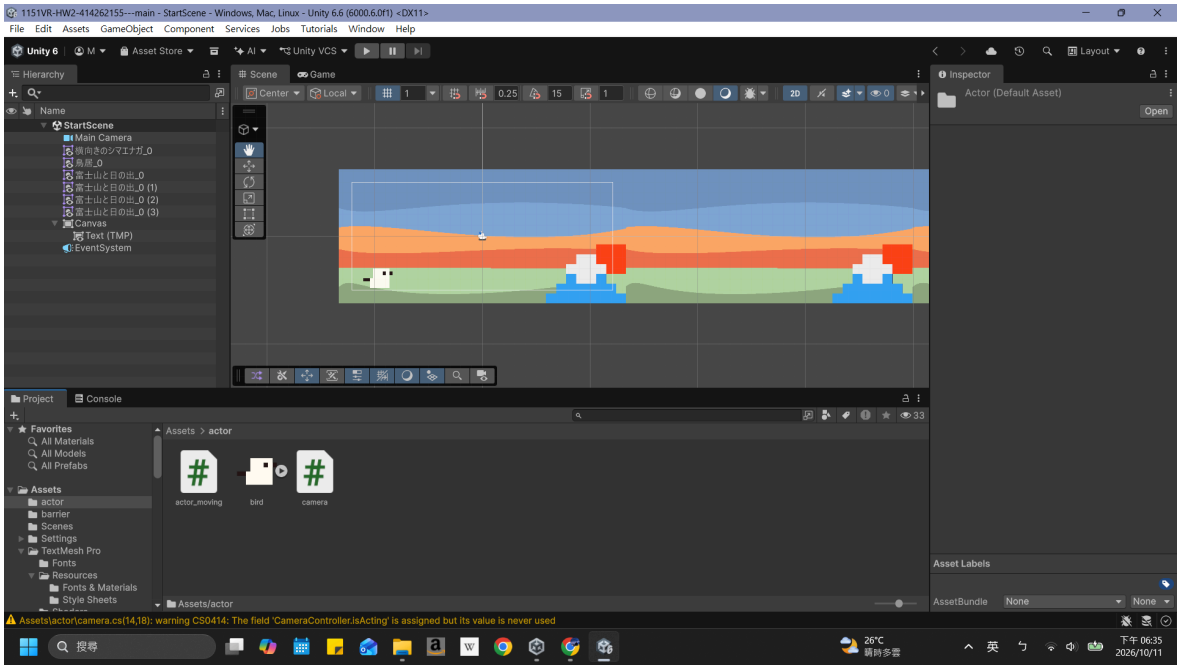
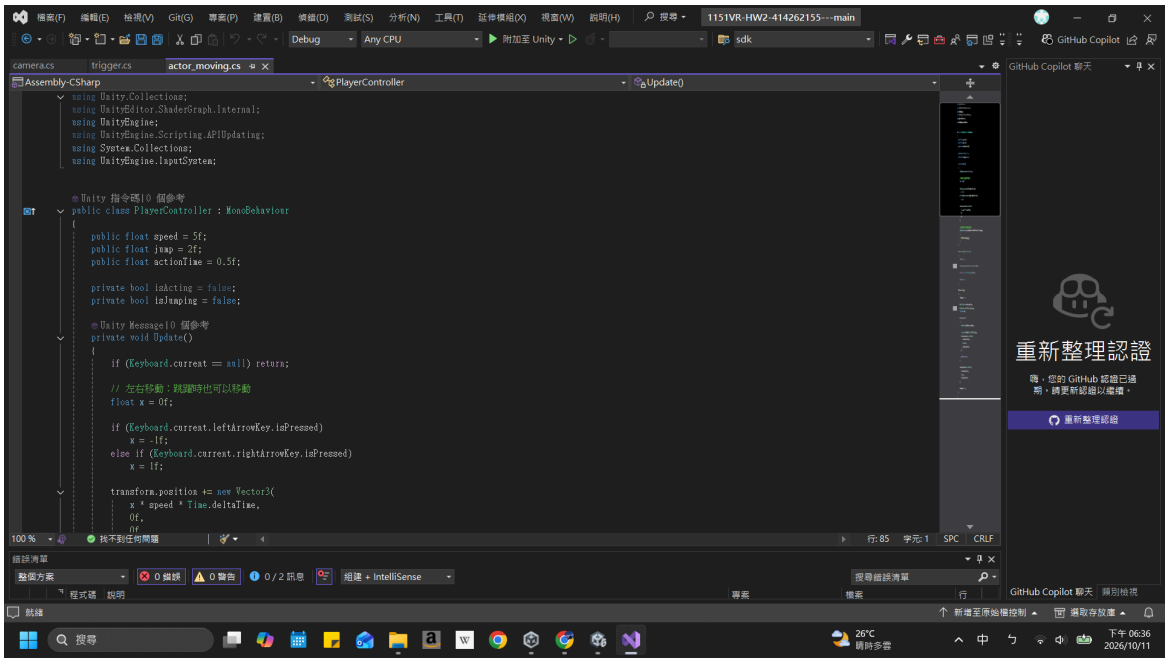
1151VR-HW2-414262155-戴泊如
Unity截圖

Visual Studio截圖


GitHub	https://github.com/Miling-000/1151VR-HW2-414262155--.git
YouTube	https://youtu.be/nA3hGTo0i1Y

製作流程	1. 背景、角色： a. 2D圖形下載 b. 調整大小和位置：背景圖2X*4(layer=0)、小鳥0.5X(layer=1)、鳥居1X(layer=1) 2. player(小鳥)： a. C# : 使用Visual Studio製作player的mono behaviour i. 宣告Speed、JumpHeight(跳的最高高度)、ActionTime(動作執行時間) ii. 宣告vector directions依序為：左、右、上 iii. 宣告 bool 的執行狀態：isActing、isJumping iv. 宣告執行動作的private function : Update v. (Update內)依不同按鍵有不同行動：往左為負數;往右為正數，依此判斷使用陣列中的哪個方向、或是跳 vi. 宣告Jump function： 1. 因為有跳高落地的過程，運用time確認上升、下降時間 2. 用Time.delta / ActionTime 決定這一單位時間該移動多少 3. 用transform.position確認回到原高度y=0 b. add component： i. Box Collider 2D：為接觸終點 ii. Rigidbody 2D：為接觸終點(gravity設為0) 3. 終點(鳥居)： a. C# : 使用Visual Studio製作終點的mono behaviour i. 開始遊戲時不顯示遊戲結束的提示 gameOverText.SetActive(false)文字預設為不顯示 ii. 宣告OnTriggerEnter2D function去檢測player是否碰到終點object iii. gameOverText.SetActive(true)文字顯示 iv. 結束遊戲 b. 新增結束的text在hierarchy c. add component : Box Collider 2D : 設定為Is Trigger(作為結束的object)、將結束的text拉入其中的Game Over Text 4. Camera： a. C# : 使用Visual Studio製作player的mono behaviour i. 移動方式參造player但速度-1(為體現出player在移動的相對速度差) ii. 設81f為邊界(及player抵達邊界的大致位置)避免看到破圖

<table>
<tr><td>過程優化/
更動</td><td>
<table>
<tr><td>移動方式之IsActing原為按下一次按鍵為一次動作導致看起來卡頓、操作手感不佳</td><td>改為按下按鍵即動作(運用Keyboard.current)、一直按著即不斷動作</td></tr>
<tr><td>操作方式的按鍵本來將跳的動作並在IsActing的執行狀態內，導致無法同時在左右移動時跳躍</td><td>將跳單獨分出來一個狀態(IsJumping)、獨立為一個狀態，使可以向左跳或向右跳、而非原地上下(保留跳只能在落地結束狀態後再次起跳)</td></tr>
<tr><td>camera原為按照player的程式和移動方式(去除跳的部分)導致兩者無相對速度差，看不出player的移動</td><td>camera降速並設有邊界，避免抵達終點前player仍在移動時，camera範圍先超出背景邊緣</td></tr>
<tr><td>移動距離為5f一次，導致每次移動砍起來卡頓視覺效果不好</td><td>改為1f並速度*Time.deltaTime更多細節移動</td></tr>
</table>
</td></tr>
<tr><td>相關操作</td><td>方向鍵操作：
◀️：左
▶️：右
空白鍵：跳

持續往右走，走到鳥居(終點)處結束。</td></tr>
</table>